МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Алгоритмы и структуры данных

Лабораторная работа №4. Бинарные деревья

Руководитель _____ В.В. Тынченко
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151438 _____ Д.А, Пузынин
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

- изучение метода рекурсивной обработки данных.
- знакомство с принципами организации и обработки бинарных деревьев

ПОСТАНОВКА ЗАДАЧИ

Постановка задачи.
Выполнить задание в соответствии с вариантом. Для решения поставленной задачи разработать и использовать шаблон класса «Бинарное дерево поиска».

Для оценки «удовлетворительно». Шаблон класса «Бинарное дерево поиска» должен включать в себя необходимый минимум методов, обеспечивающий полноценное функционирование объектов указанного класса, а именно:

- конструктор по умолчанию;
- конструктор копирования;
- деструктор;
- добавление узла;
- удаление узла;
- поиск узла с заданным значением ключа;

Используя механизм перегрузки стандартных операций, реализовать:

- вывод дерева на экран в удобном для восприятия человеком виде;
- ввод данных в дерево с клавиатуры;
- сравнение двух деревьев;
- присваивание данных одного дерева другому дереву;
- решение задачи согласно варианту задания.

Для оценки «хорошо». Дополнить разработанный класс следующими функциональными возможностями:

- прямой обход дерева (корень, левое поддерево, правое поддерево);
- обратный обход дерева (левое поддерево, корень, правое поддерево);
- концевой обход дерева (левое поддерево, правое поддерево, корень).
- вывод дерева в файл в удобном для восприятия человеком виде;
- ввод данных в дерево из файла;

Для оценки «отлично». На базе разработанного класса «Бинарное дерево поиска», реализовать производный класс «Сбалансированное бинарное дерево», добавив показатель балансировки для узлов дерева и метод, выполняющий при необходимости повторную балансировку дерева.

Вариант №13.
Из входной последовательности вещественных чисел построить бинарное дерево поиска. Вычислить среднее арифметическое всех узлов дерева. Вывести на экран те значения в узлах дерева, которые больше среднего арифметического, а также номер уровня для каждого такого узла.

ХОД РАБОТЫ

В ходе работы была разработана программа для организации и обработки бинарных деревьев поиска.

Была реализована структура ***Node*** (Рисунок 1), которая хранит значение узла и указатели на левое и правое поддеревья, а также поле ***height*** для балансировки.

На основе структуры *Node* был создан класс ***Tree*** (Рисунок 2), обеспечивающий:

- хранение корня дерева,
- создание и удаление дерева,
- добавление и удаление узлов,
- поиск узлов,
- обход дерева прямым, обратным и концевым методами,
- вывод дерева на экран и в файл,
- ввод данных с клавиатуры и из файла,
- вычисление среднего арифметического значений узлов и выделение узлов, больших среднего.

Разработаны конструкторы по умолчанию, с параметрами, копирования. деструктор (Рисунок 1), (Рисунок 3), (Рисунок 5). Также были реализованы методы добавления узла, удаления узла, поиск узла с заданным значением ключа (Рисунок 3).

```
struct Node {
    double key;
    Node* left;
    Node* right;
    int height; // для сбалансированного дерева
    Node(double k) : key(k), left(nullptr), right(nullptr), height(1) {}
};
```

Рисунок 1 – Структура Node

```
class Tree {
protected:
    Node* root;
```

Рисунок 2 – Класс Tree

```
21      // Копирование дерева рекурсивно
22      Node* copy(Node* node) {
23          if (!node) return nullptr;
24          Node* newNode = new Node(node->key);
25          newNode->left = copy(node->left);
26          newNode->right = copy(node->right);
27          newNode->height = node->height;
28          return newNode;
29      }
30
31      // Очистка дерева рекурсивно
32      void clear(Node* node) {
33          if (!node) return;
34          clear(node->left);
35          clear(node->right);
36          delete node;
37      }
38
39      // Вставка узла рекурсивно
40      virtual Node* add(Node* node, double key) {
41          if (!node) return new Node(key);
42          if (key < node->key) node->left = add(node->left, key);
43          else node->right = add(node->right, key);
44          return node;
45      }
46
47      // Поиск узла
48      Node* search(Node* node, double key) {
49          if (!node) return nullptr;
50          if (node->key == key) return node;
51          if (key < node->key) return search(node->left, key);
52          else return search(node->right, key);
53      }
54
55      // Поиск минимального узла
56      Node* findMin(Node* node) {
57          while (node && node->left) node = node->left;
58          return node;
59      }
60
61      // Удаление узла
62      Node* remove(Node* node, double key) {
63          if (!node) return nullptr;
64          if (key < node->key) node->left = remove(node->left, key);
65          else if (key > node->key) node->right = remove(node->right, key);
66          else {
67              if (!node->left) {
68                  Node* temp = node->right;
69                  delete node;
70                  return temp;
71              } else if (!node->right) {
72                  Node* temp = node->left;
73                  delete node;
74                  return temp;
75              }
76              Node* temp = findMin(node->right);
77              node->key = temp->key;
78              node->right = remove(node->right, temp->key);
79          }
80          return node;
81      }
82
83      // Прямой обход
84      void pre(Node* node) {
85          if (!node) return;
86          cout << node->key << " ";
87          pre(node->left);
88          pre(node->right);
89      }
90
```

Рисунок 3 – Методы класса Tree (1/2)

```
91      // Обратный обход
92      void in(Node* node) {
93          if (!node) return;
94          in(node->left);
95          cout << node->key << " ";
96          in(node->right);
97      }
98
99      // Концевой обход
100     void post(Node* node) {
101         if (!node) return;
102         post(node->left);
103         post(node->right);
104         cout << node->key << " ";
105     }
106
107     // Вывод дерева
108     void print(Node* node, int indent) {
109         if (!node) return;
110         if (node->right) print(node->right, indent + 4);
111         if (indent) cout << setw(indent) << ' ';
112         cout << node->key << "\n";
113         if (node->left) print(node->left, indent + 4);
114     }
115
116     // Сбор всех значений для среднего
117     void collectValues(Node* node, vector<double>& vals) {
118         if (!node) return;
119         vals.push_back(node->key);
120         collectValues(node->left, vals);
121         collectValues(node->right, vals);
122     }
123
124     // Сбор узлов больше среднего
125     void collectGreater(Node* node, double avg, int level, vector<pair<double,int>>& result) {
126         if (!node) return;
127         if (node->key > avg) result.push_back({node->key, level});
128         collectGreater(node->left, avg, level + 1, result);
129         collectGreater(node->right, avg, level + 1, result);
130     }
131
132     // Вспомогательная функция для сравнения деревьев
133     bool compare(Node* a, Node* b) const {
134         if (!a && !b) return true;
135         if (!a || !b) return false;
136         return (a->key == b->key) &&
137                compare(a->left, b->left) &&
138                compare(a->right, b->right);
139     }
140
```

Рисунок 4 – Методы класса Tree (2/2)

copy – выполняет рекурсивное копирование бинарного дерева. Для каждого узла создаётся новый объект, а его потомки копируются тем же способом.

clear – рекурсивно очищает дерево, удаляя все узлы и освобождая память.

add – рекурсивно вставляет новый узел с заданным значением в бинарное дерево поиска, сохраняя порядок (левое поддерево меньше, правое – больше).

search – выполняет рекурсивный поиск узла по заданному значению ключа и возвращает указатель на найденный элемент.

findMin – находит узел с минимальным значением в дереве, проходя по левым потомкам до конца.

remove – рекурсивно удаляет узел с указанным ключом. При необходимости перестраивает связи, чтобы сохранить корректность структуры дерева.

pre – выполняет прямой (префиксный) обход дерева: сначала корень, затем левое и правое поддеревья.

in – выполняет обратный (инфиксный) обход дерева: сначала левое поддерево, затем корень, потом правое.

post – выполняет концевой (постфиксный) обход дерева: сначала левое и правое поддеревья, потом корень.

print – выводит дерево на экран в наглядном виде с отступами, показывая структуру уровней.

collectValues – собирает все значения узлов дерева в вектор для последующего вычисления среднего арифметического.

collectGreater – рекурсивно проходит по дереву и добавляет в список пары (значение, уровень) для узлов, значения которых больше среднего арифметического.

compare – выполняет рекурсивное сравнение двух деревьев, проверяя совпадение значений и структуры узлов.

В данном фрагменте кода (Рисунок 5) реализована **публичная часть** класса *Tree*, отвечающая за взаимодействие пользователя с бинарным деревом поиска.

Здесь определены конструкторы, деструктор, перегрузки операторов, а также функции для добавления, удаления, поиска, обходов дерева, работы с файлами и вычисления среднего арифметического.

```
141 public:
142     Tree() : root(nullptr) {}
143     Tree(const Tree& other) { root = copy(other.root); }
144     virtual ~Tree() { clear(root); }
145
146     // Перегрузка оператора присваивания
147     Tree& operator=(const Tree& other) {
148         if (this != &other) {
149             clear(root);
150             root = copy(other.root);
151         }
152         return *this;
153     }
154
155     // Перегрузка оператора сравнения
156     bool operator==(const Tree& other) const {
157         return compare(root, other.root);
158     }
159
160     // Интерфейсные функции
161     void addValue(double key) { root = add(root, key); }  // публичная обёртка для вставки
162     void removeValue(double key) { root = remove(root, key); }
163     bool searchValue(double key) { return search(root, key) != nullptr; }
164     void printTree() { print(root, 0); }
165     void preOrder() { pre(root); cout << endl; }
166     void inOrder() { in(root); cout << endl; }
167     void postOrder() { post(root); cout << endl; }
168
169     void saveToFile(const string& filename) {
170         ofstream out(filename);
171         if (!out.is_open()) return;
172         saveHelper(root, out, 0);
173         out.close();
174     }
175
176     void saveHelper(Node* node, ofstream& out, int level) {
177         if (!node) return;
178         out << string(level*4,' ') << node->key << "\n";
179         saveHelper(node->left, out, level+1);
180         saveHelper(node->right, out, level+1);
181     }
182
183     void loadFromFile(const string& filename) {
184         ifstream in(filename);
185         double val;
186         while (in >> val) addValue(val);
187         in.close();
188     }
189
190     double average() {
191         vector<double> vals;
192         collectValues(root, vals);
193         if (vals.empty()) return 0;
194         double sum = 0;
195         for (double v : vals) sum += v;
196         return sum / vals.size();
197     }
198
199     void showGreaterThanAverage() {
200         double avg = average();
201         vector<pair<double,int>> result;
202         collectGreater(root, avg, 1, result);
203         cout << "Среднее арифметическое: " << avg << "\n";
204         for (auto& p : result)
205             cout << "Значение: " << p.first << ", уровень: " << p.second << "\n";
206     }
207 };
```

Рисунок 5 – Публичные методы класса Tree

***preOrder*()** – выполняет **прямой обход** дерева (корень → левое поддерево → правое поддерево).

***inOrder*()** – выполняет **обратный (симметричный) обход** дерева (левое поддерево → корень → правое поддерево).

***postOrder*()** – выполняет **концевой обход** дерева (левое поддерево → правое поддерево → корень).

***saveToFile*(*const string& filename*)** – сохраняет текущее дерево в файл в наглядном виде, вызывая вспомогательную функцию *saveHelper*().

saveHelper(Node node, ofstream& out, int level) – рекурсивная функция для записи дерева в файл. Каждый уровень дерева отображается отступом для удобства восприятия.

***loadFromFile*(*const string& filename*)** – загружает значения узлов дерева из указанного файла, восстанавливая структуру дерева.

***average*()** – вычисляет среднее арифметическое значений всех узлов дерева. Собирает их в вектор и находит сумму и количество.

***showGreaterThanAverage*()** – вычисляет среднее арифметическое и выводит значения узлов, **превышающие это среднее**, а также **номер уровня**, на котором они находятся.

Для реализации сбалансированного бинарного дерева был создан производный класс **AVL**:

```
class AVL : public Tree {
    int height(Node* node) { return node ? node->height : 0; }
    int balance(Node* node) { return node ? height(node->left)-height(node->right) : 0; }

    Node* rotateRight(Node* y) {
        Node* x = y->left;
        Node* T2 = x->right;
        x->right = y;
        y->left = T2;
        y->height = max(height(y->left), height(y->right))+1;
        x->height = max(height(x->left), height(x->right))+1;
        return x;
    }

    Node* rotateLeft(Node* x) {
        Node* y = x->right;
        Node* T2 = y->left;
        y->left = x;
        x->right = T2;
        x->height = max(height(x->left), height(x->right))+1;
        y->height = max(height(y->left), height(y->right))+1;
        return y;
    }

    Node* addAVL(Node* node, double key) {
        if (!node) return new Node(key);
        if (key < node->key) node->left = addAVL(node->left, key);
        else node->right = addAVL(node->right, key);

        node->height = max(height(node->left), height(node->right))+1;
        int bf = balance(node);

        if (bf > 1 && key < node->left->key) return rotateRight(node);
        if (bf < -1 && key > node->right->key) return rotateLeft(node);
        if (bf > 1 && key > node->left->key) { node->left = rotateLeft(node->left); return rotateRight(node);}
        if (bf < -1 && key < node->right->key) { node->right = rotateRight(node->right); return rotateLeft(node);}
        return node;
    }

protected:
    Node* add(Node* node, double key) override {
        return addAVL(node, key);
    }
};
```

Рисунок 6 – Производный класс AVL

Класс AVL реализует самобалансирующееся бинарное дерево поиска (AVL-дерево). После каждой вставки элементов дерево автоматически поддерживает баланс, чтобы высота левого и правого поддерева любого узла отличалась не более чем на 1.

int height(Node node) – возвращает высоту указанного узла. Если узел отсутствует, возвращает 0.

int balance(Node node) – вычисляет баланс узла как разность высот левого и правого поддерева.

***Node rotateRight(Node* y)** – выполняет правый поворот поддерева для восстановления баланса, когда левое поддерево слишком высокое.

***Node rotateLeft(Node* x)** – выполняет левый поворот поддерева, если правое поддерево имеет большую высоту.

***Node addAVL(Node* node, double key)** – рекурсивная функция вставки нового узла с учётом балансировки.

После добавления элемента вычисляется фактор баланса (bf), и при необходимости выполняются соответствующие повороты (левый, правый или двойной).

***Node add(Node node, double key*)** – переопределяет метод базового класса *Tree*. Реализует добавление узла с использованием алгоритма AVL-вставки (*addAVL*), обеспечивая балансировку дерева.

```
void showMenu() {
    cout << "\n============== МЕНЮ ==============\n";
    cout << "1. Добавить узел (ввод с клавиатуры)\n";
    cout << "2. Удалить узел\n";
    cout << "3. Найти узел\n";
    cout << "4. Вывести дерево (визуально)\n";
    cout << "5. Обходы дерева (Прямой, Обратный, Концевой)\n";
    cout << "6. Среднее и фильтр\n";
    cout << "7. Сохранить в файл\n";
    cout << "8. Загрузить из файла\n";
    cout << "9. Тест операторов (= и ==)\n";
    cout << "10. Использовать operator>> (ввод одного значения)\n";
    cout << "0. Выход\n";
    cout << "==================================\n";
    cout << "Ваш выбор: ";
}
```

Рисунок 7 – Меню программы (1/2)

```
    switch (choice) {
    case 1:
        cout << "Введите значение (double): ";
        cin >> val;
        tree.addValue(val);
        cout << "Узел добавлен.\n";
        break;
    case 2:
        cout << "Введите значение для удаления: ";
        cin >> val;
        tree.removeValue(val);
        cout << "Команда удаления выполнена (если узел существовал).\n";
        break;
    case 3:
        cout << "Введите значение для поиска: ";
        cin >> val;
        if (tree.searchValue(val)) cout << "Узел НАЙДЕН.\n";
        else cout << "Узел НЕ найден.\n";
        break;
    case 4:
        cout << "\n--- Текущее дерево (operator<<) ---\n";
        cout << tree; // Использование перегруженного оператора <<
        break;
    case 5:
        cout << "Прямой обход (PreOrder): "; tree.preOrder();
        cout << "Обратный обход (InOrder): "; tree.inOrder();
        cout << "Концевой обход (PostOrder): "; tree.postOrder();
        break;
    case 6:
        tree.executeVariantTask();
        break;
    case 7:
        cout << "Введите имя файла: ";
        cin >> filename;
        tree.saveToFile(filename);
        break;
    case 8:
        cout << "Введите имя файла: ";
        cin >> filename;
        tree.loadFromFile(filename);
        break;
    case 9:
        cout << "\n--- Тестирование операторов ---\n";
        testTree = tree; // Тест operator=
        cout << "Дерево скопировано в testTree через operator=.\n";
        cout << "Сравнение tree == testTree: " << (tree == testTree ? "РАВНЫ" : "НЕ РАВНЫ") << endl;

        cout << "Добавим 999.9 в testTree...\n";
        testTree.addValue(999.9);
        cout << "Сравнение tree == testTree: " << (tree == testTree ? "РАВНЫ" : "НЕ РАВНЫ") << endl;
        break;
    case 10:
        // Тест operator>>
        cin >> tree;
        break;
    case 0:
        return 0;
    default:
        cout << "Неверный выбор. Попробуйте снова.\n";
    }
}
```

Рисунок 8 - Меню программы (2/2)

Пример работы программы:

```
============== МЕНЮ ==============
1. Добавить узел (ввод с клавиатуры)
2. Удалить узел
3. Найти узел
2. Удалить узел
3. Найти узел
4. Вывести дерево (визуально)
5. Обходы дерева (Прямой, Обратный, Концевой)
6. Среднее и фильтр
7. Сохранить в файл
8. Загрузить из файла
9. Тест операторов (= и ==)
10. Использовать operator>> (ввод одного значения)
0. Выход
==================================
Ваш выбор: 4

--- Текущее дерево (operator<<) ---
        15.000
    11.000
10.000
        5.000
    0.000
```

Рисунок 9 – Вывод дерева

```
Ваш выбор: 1
Введите значение (double): 51.52
Узел добавлен.

============== МЕНЮ ==============
1. Добавить узел (ввод с клавиатуры)
2. Удалить узел
3. Найти узел
4. Вывести дерево (визуально)
5. Обходы дерева (Прямой, Обратный, Концевой)
6. Среднее и фильтр
7. Сохранить в файл
8. Загрузить из файла
9. Тест операторов (= и ==)
10. Использовать operator>> (ввод одного значения)
0. Выход
==================================
Ваш выбор: 2
Введите значение для удаления: 10
Команда удаления выполнена (если узел существовал).

============== МЕНЮ ==============
1. Добавить узел (ввод с клавиатуры)
2. Удалить узел
3. Найти узел
4. Вывести дерево (визуально)
5. Обходы дерева (Прямой, Обратный, Концевой)
6. Среднее и фильтр
7. Сохранить в файл
8. Загрузить из файла
9. Тест операторов (= и ==)
10. Использовать operator>> (ввод одного значения)
0. Выход
==================================
Ваш выбор: 4

--- Текущее дерево (operator<<) ---
        51.520
    15.000
11.000
        5.000
    0.000
```

Рисунок 10 - Добавление и удаление узла

```
Ваш выбор: 3
Введите значение для поиска: 5
Узел НАЙДЕН.

============== МЕНЮ ==============
1. Добавить узел (ввод с клавиатуры)
2. Удалить узел
3. Найти узел
4. Вывести дерево (визуально)
5. Обходы дерева (Прямой, Обратный, Концевой)
6. Среднее и фильтр
7. Сохранить в файл
8. Загрузить из файла
9. Тест операторов (= и ==)
10. Использовать operator>> (ввод одного значения)
0. Выход
==================================
Ваш выбор: 3
Введите значение для поиска: 5111
Узел НЕ найден.
```

Рисунок 11 - Поиск узла

```
Ваш выбор: 5
Прямой обход (PreOrder): 11.000 0.000 5.000 15.000 51.520
Обратный обход (InOrder): 0.000 5.000 11.000 15.000 51.520
Концевой обход (PostOrder): 5.000 0.000 51.520 15.000 11.000
```

Рисунок 12 - Обходы дерева

```
Ваш выбор: 6
Среднее арифметическое: 16.504
Узлы со значением больше среднего:
Значение | Уровень
  51.520 | 3
```

Рисунок 13 - Среднее арифметическое и фильтр

```
Ваш выбор: 7
Введите имя файла: output
Дерево сохранено в файл output
```

Рисунок 14 - Сохранение в файл

```
C_codespace > 2 kurs > 3-rd senester > Algorithms and data structure > Laboratory work > Laba #4 > Dima > output
  1    11 0 5 15 51.52
```

Рисунок 15 - Файл с сохраненными данными

```
Ваш выбор: 8
Введите имя файла: output
Дерево загружено из файла output
```

Рисунок 16 - Загрузка из файла

```
Ваш выбор: 9

--- Тестирование операторов ---
Дерево скопировано в testTree через operator=.
Сравнение tree == testTree: РАВНЫ
Добавим 999.9 в testTree...
Сравнение tree == testTree: НЕ РАВНЫ
```

Рисунок 17 - Тест операторов

```
Ваш выбор: 10
Введите значение для добавления: 151

============== МЕНЮ ==============
1. Добавить узел (ввод с клавиатуры)
2. Удалить узел
3. Найти узел
4. Вывести дерево (визуально)
5. Обходы дерева (Прямой, Обратный, Концевой)
6. Среднее и фильтр
7. Сохранить в файл
8. Загрузить из файла
9. Тест операторов (= и ==)
10. Использовать operator>> (ввод одного значения)
0. Выход
==================================
Ваш выбор: 4

--- Текущее дерево (operator<<) ---
        151.000
    51.520
15.000
        11.000
    5.000
        0.000
```

Рисунок 18 - Использование operator>>

ВЫВОД

В ходе выполнения лабораторной работы была реализована программа, моделирующая работу бинарного дерева поиска и его модификации – сбалансированного AVL-дерева. Были изучены основные операции над деревом: добавление, удаление, поиск, обход и балансировка узлов. Полученные знания позволили закрепить принципы построения и работы динамических структур данных, а также понять преимущества самобалансирующихся деревьев при хранении и обработке информации.